AgroSmart

Análisis de Algoritmos y Estructuras de Datos

Módulo: Clima y Alertas

1. Sistema de Cache con Diccionario

Explicación del funcionamiento

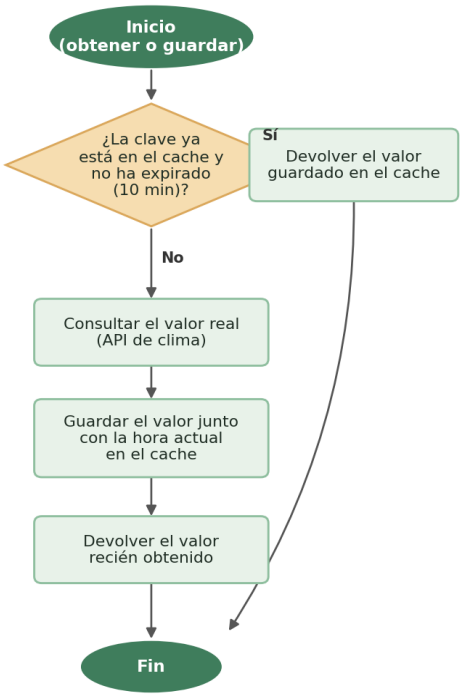
El módulo de clima guarda en memoria, dentro de un diccionario (_CACHE), el último resultado obtenido para cada localidad. Cuando se pide el clima de una localidad, primero se revisa si ya existe un resultado guardado y si no han pasado más de 10 minutos desde que se guardó; si es así, se devuelve ese valor de inmediato sin volver a consultar la API. Si no existe o ya expiró, se consulta el valor real, se guarda en el diccionario junto con la hora en que se guardó, y se devuelve. El diccionario de Python funciona internamente como una tabla hash: usa la clave (por ejemplo, el nombre de la localidad) para ubicar el valor de forma casi instantánea, sin recorrer toda la estructura.

Pseudocódigo

```
FUNCION obtener_de_cache(clave)
  SI clave EXISTE en cache ENTONCES
    (valor, guardado_en) = cache[clave]
    SI (tiempo_actual - guardado_en) < duracion_cache ENTONCES
      DEVOLVER valor
    FIN SI
  FIN SI
  DEVOLVER nulo
FIN FUNCION

FUNCION guardar_en_cache(clave, valor)
  cache[clave] = (valor, tiempo_actual)
FIN FUNCION
```

Diagrama de flujo



Análisis de complejidad

Complejidad temporal	O(1) en promedio — el diccionario calcula directamente la posición de la clave (localidad) sin recorrer los demás datos guardados.
Complejidad espacial	O(k) — donde k es el número de localidades distintas consultadas; se guarda una entrada por cada una.

Justificación de su elección

Se eligió un cache basado en diccionario porque las consultas al clima dependen de un servicio externo (Open-Meteo) que puede ser lento o tener límites de uso. Guardar temporalmente el último resultado evita hacer peticiones repetidas cuando varios integrantes del equipo prueban el programa en poco tiempo, y el diccionario nativo de Python es la estructura más eficiente para este tipo de búsqueda por clave.

2. Recorrido Lineal para el Pronóstico de Varios Días

Explicación del funcionamiento

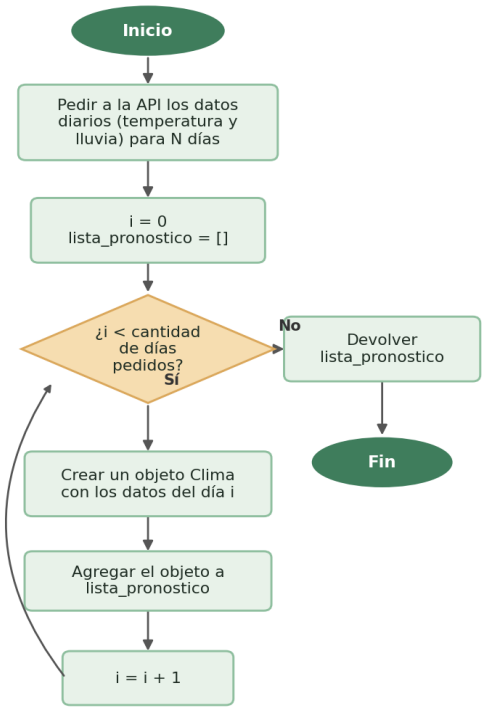
Para construir el pronóstico de varios días, el programa recibe de la API una lista con los datos de cada día (temperatura máxima, probabilidad de lluvia y código de clima) y recorre esa lista una posición a la vez, de principio a fin. En cada posición crea un objeto Clima con los datos de ese día y lo agrega a una nueva lista de pronóstico. Este recorrido se repite hasta procesar todos los días pedidos.

Pseudocódigo

```
FUNCION obtener_pronostico_dias(localidad, dias)
  datos_diarios = PEDIR_A_LA_API(localidad, dias)
  lista_pronostico = []
  PARA i DESDE 0 HASTA dias - 1
    clima_dia = CREAR_CLIMA(datos_diarios[i])
```

```
    AGREGAR clima_dia A lista_pronostico
  FIN PARA
  DEVOLVER lista_pronostico
FIN FUNCION
```

Diagrama de flujo



Análisis de complejidad

Complejidad temporal	O(n) — se revisa cada uno de los n días del pronóstico exactamente una vez.
Complejidad espacial	O(n) — se crea una lista nueva con un objeto Clima por cada día procesado.

Justificación de su elección

Se usó un recorrido lineal simple porque los datos que devuelve la API ya vienen en orden (del día más cercano al más lejano) y la cantidad de días de un pronóstico es siempre pequeña (entre 1 y 16 días), por lo que no se necesita ningún algoritmo de búsqueda u ordenamiento adicional: basta con procesar cada día una sola vez.

3. Árbol de Decisión para Generar Alertas

Explicación del funcionamiento

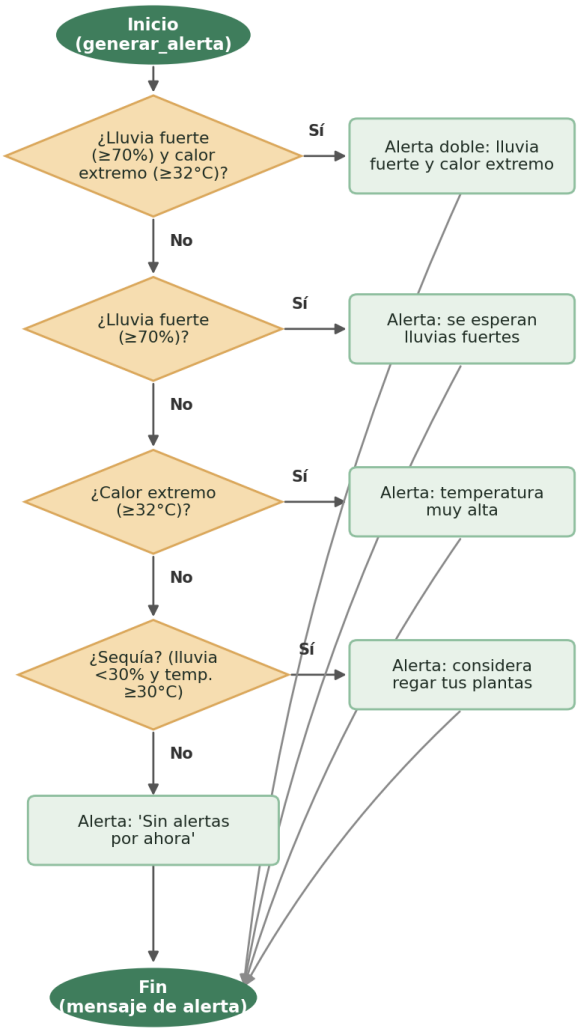
La función generar_alerta no recorre datos ni busca elementos: evalúa una serie de condiciones en un orden fijo (lluvia fuerte y calor extremo, luego solo lluvia fuerte, luego solo calor extremo, luego sequía) y se detiene en la primera condición que se cumple, devolviendo el mensaje de alerta correspondiente. Si ninguna condición se cumple, devuelve el mensaje de que no hay alertas. Este tipo de lógica se conoce como árbol de decisión, porque en cada paso se toma un camino distinto (Sí o No) según el resultado de la comparación.

Pseudocódigo

```
FUNCION generar_alerta(temperatura, probabilidad_lluvia)
  lluvia_fuerte = probabilidad_lluvia >= 70
```

```
  calor_extremo = temperatura >= 32
  sequia = probabilidad_lluvia < 30 Y temperatura >= 30
  SI lluvia_fuerte Y calor_extremo ENTONCES
    DEVOLVER 'Alerta doble: lluvias fuertes y temperatura muy alta'
  SINO SI lluvia_fuerte ENTONCES
    DEVOLVER 'Alerta: se esperan lluvias fuertes'
  SINO SI calor_extremo ENTONCES
    DEVOLVER 'Alerta: temperatura muy alta'
  SINO SI sequia ENTONCES
    DEVOLVER 'Alerta: considera regar tus plantas'
  SINO
    DEVOLVER 'Sin alertas por ahora'
  FIN SI
FIN FUNCION
```

Diagrama de flujo



Análisis de complejidad

Complejidad temporal	O(1) — sin importar los valores de entrada, siempre se evalúa como máximo un número fijo de condiciones (4), nunca crece con la cantidad de datos.
Complejidad espacial	O(1) — solo se usan unas pocas variables booleanas temporales, sin estructuras adicionales.

Justificación de su elección

Se usó una cascada de condiciones ordenadas de la más urgente (lluvia fuerte y calor extremo juntos) a la menos urgente (sequía), en lugar de evaluar todas por separado, para que el agricultor reciba un solo mensaje claro con el problema más importante en lugar de varias alertas encimadas. Al ser un número fijo y pequeño de condiciones, no se necesitó ninguna estructura de datos adicional ni un algoritmo de búsqueda.

4. Ordenamiento por Inserción

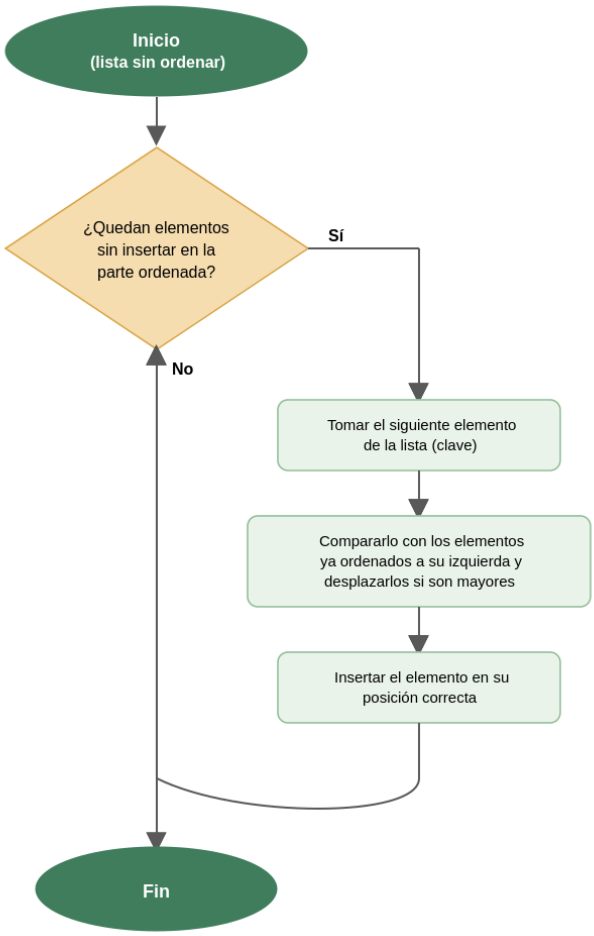
Explicación del funcionamiento

Este algoritmo ordena el pronóstico de varios días (por ejemplo, por temperatura o por probabilidad de lluvia) construyendo la lista ordenada de a un elemento a la vez. Se toma cada elemento de la lista, comenzando por el segundo, y se compara con los elementos que ya están ordenados a su izquierda, desplazándolos una posición a la derecha mientras sean mayores que el elemento actual, hasta encontrar el lugar donde debe insertarse. Este proceso se repite hasta que todos los elementos han sido insertados en su posición correcta.

Pseudocódigo

```
FUNCION ordenamiento_insercion(lista_climas, clave)
  PARA i DESDE 1 HASTA longitud(lista_climas) - 1
    actual = lista_climas[i]
    j = i - 1
    MIENTRAS j >= 0 Y lista_climas[j].clave > actual.clave
      lista_climas[j + 1] = lista_climas[j]
      j = j - 1
    FIN MIENTRAS
    lista_climas[j + 1] = actual
  FIN PARA
  DEVOLVER lista_climas
FIN FUNCION
```

Diagrama de flujo



Análisis de complejidad

Complejidad temporal	O(n²) en el peor caso (lista en orden inverso) y O(n) en el mejor caso (lista ya ordenada) — por cada uno de los n elementos, puede ser necesario desplazar hasta n-1 elementos ya ordenados.
Complejidad espacial	O(1) — el ordenamiento se hace in-place, desplazando elementos dentro de la misma lista, sin crear estructuras auxiliares adicionales.

Justificación de su elección

Se usó ordenamiento por inserción por ser un algoritmo simple de implementar y fácil de explicar, adecuado para listas pequeñas como el pronóstico de 5 días que maneja este módulo. Al tratarse de una cantidad de datos reducida, su complejidad O(n²) en el peor caso no representa un problema real de rendimiento para este caso de uso.

5. Ordenamiento por Mezcla (Merge Sort)

Explicación del funcionamiento

Como segundo algoritmo de ordenamiento, se implementó Merge Sort (ordenamiento por mezcla), que sigue la estrategia de divide y vencerás. La lista de días del pronóstico se divide repetidamente a la mitad hasta llegar a sublistas de un solo elemento (que por definición ya están ordenadas). Luego, esas sublistas se van combinando (mezclando) de dos en dos: en cada mezcla se comparan los elementos de ambas mitades según el atributo elegido (temperatura o probabilidad_lluvia) y se van colocando en orden en una nueva lista, hasta reconstruir la lista completa ya ordenada.

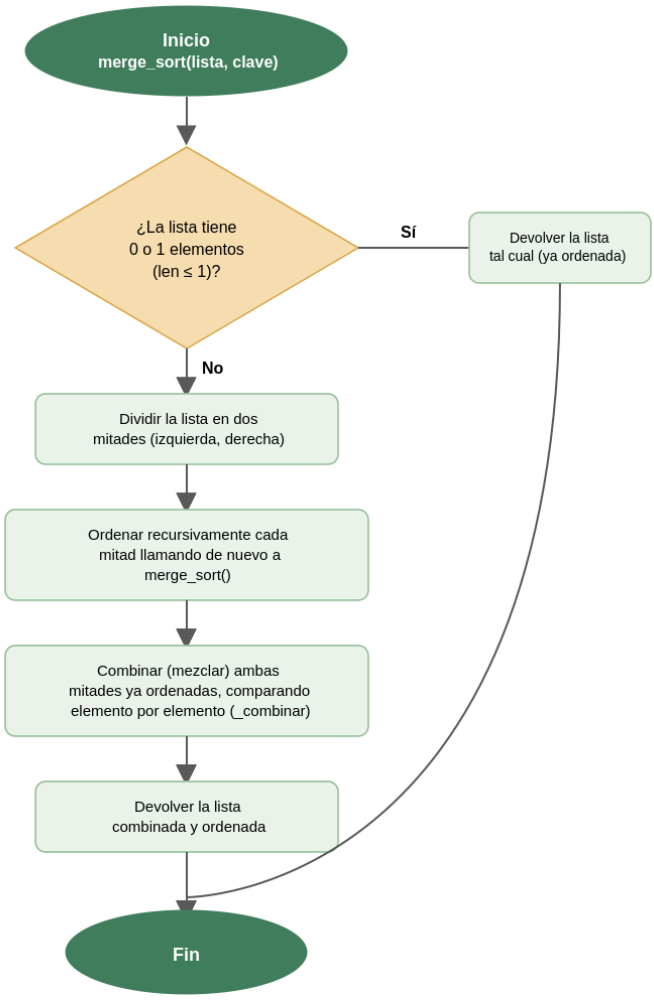
Pseudocódigo

```
FUNCION merge_sort(lista_climas, clave)
```

```
  SI longitud(lista_climas) <= 1 ENTONCES
    DEVOLVER lista_climas
  FIN SI
  medio = longitud(lista_climas) // 2
  izquierda = merge_sort(lista_climas[:medio], clave)
  derecha = merge_sort(lista_climas[medio:], clave)
  DEVOLVER _combinar(izquierda, derecha, clave)
FIN FUNCION

FUNCION _combinar(izquierda, derecha, clave)
  resultado = []
  i = 0, j = 0
  MIENTRAS i < longitud(izquierda) Y j < longitud(derecha)
    SI izquierda[i].clave <= derecha[j].clave ENTONCES
      AGREGAR izquierda[i] A resultado; i = i + 1
    SINO
      AGREGAR derecha[j] A resultado; j = j + 1
    FIN SI
  FIN MIENTRAS
  AGREGAR el resto de izquierda (si queda) A resultado
  AGREGAR el resto de derecha (si queda) A resultado
  DEVOLVER resultado
FIN FUNCION
```

Diagrama de flujo



Análisis de complejidad

Complejidad temporal	O(n log n) en todos los casos (mejor, promedio y peor) — la lista se divide en log n niveles, y en cada nivel se recorren los n elementos una vez durante la mezcla.
Complejidad espacial	O(n) — en cada llamada a _combinar se crea una lista nueva (resultado) para guardar los elementos mezclados.

Justificación de su elección

Se eligió Merge Sort como segundo algoritmo porque ofrece un contraste claro frente al ordenamiento por inserción: en lugar de una estrategia incremental, usa divide y vencerás, y garantiza O(n log n) en todos los casos en vez de $O(n^2)$ en el peor caso. Esto permite comparar en el análisis ambos enfoques (uno simple pero menos eficiente en listas grandes, otro más eficiente pero con mayor uso de memoria auxiliar) usando el mismo caso de uso: ordenar el pronóstico de días por temperatura o por probabilidad de lluvia.